

Лабораторна робота 4

Теоретичні відомості

Розділ 1. Основи модуля `turtle`

Черепашка, координати та базові команди руху

Модуль `turtle` дає змогу створювати прості графічні зображення за допомогою уявної «черепашки», яка переміщується по координатній площині та залишає за собою лінію.

```
import turtle
```

Черепашка має поточні координати (x, y) та напрямок руху. За замовчуванням вона розташована в початку координат $(0, 0)$ і спрямована вздовж додатного напрямку осі x .

Основні команди керування черепашкою (використовуються у "модульному" стилі, без явного створення об'єкта):

```
turtle.forward(distance)
turtle.backward(distance)
turtle.left(angle)
turtle.right(angle)
```

де `distance` — відстань переміщення, а `angle` — кут повороту в градусах.

Для безпосереднього переміщення в задану точку використовують:

```
turtle.goto(x, y)
```

Якщо потрібно перемістити черепашку без малювання, лінію "піднімають", переміщують, а потім знову "опускають":

```
turtle.penup()
turtle.goto(x, y)
turtle.pendown()
```

Важливо (доповнення). Після завершення побудови вікно `turtle` за замовчуванням одразу закривається, тому в кінці програми слід викликати:

```
turtle.done()
```

Це утримує графічне вікно відкритим, доки користувач не закриє його вручну.

Керування виглядом лінії та тла

Товщину лінії задають командою:

```
turtle.pensize(width)
```

Колір лінії — командою:

```
turtle.pencolor(color)
```

Наприклад:

```
turtle.pencolor("blue")
```

```
turtle.pensize(3)
```

Колір тла задають так:

```
turtle.bgcolor("black")
```

Модуль `turtle` підтримує назви багатьох стандартних кольорів, наприклад `red`, `blue`, `cyan`, `orange`, `gold`, `crimson`, `turquoise`, `magenta`.

Зафарбовування замкнених фігур

Для зафарбовування замкненої фігури використовують пару команд:

```
turtle.begin_fill()
```

```
# побудова фігури
```

```
turtle.end_fill()
```

Колір заповнення встановлюють командою `turtle.fillcolor(color)`.

Наприклад:

```
turtle.fillcolor("gold")
```

```
turtle.begin_fill()
```

```
for _ in range(4):
```

```
    turtle.forward(100)
```

```
    turtle.left(90)
```

```
turtle.end_fill()
```

Доповнення. Заливка коректно працює лише тоді, коли траєкторія між `begin_fill()` і `end_fill()` утворює замкнений контур (черепашка повертається у вихідну точку); інакше фігура заливається "як є", що може виглядати не так, як очікувалося.

Побудова правильних зірок

Правильну зірку можна отримати, якщо розмістити n вершин правильного n -кутника на колі та з'єднувати їх не з найближчими сусідніми вершинами, а через певну кількість вершин. Параметр k називають **кроком з'єднання вершин**: від вершини з номером i переходять до вершини $(i + k) \bmod n$.

- при $k = 1$ кожна вершина з'єднується з найближчою сусідньою — утворюється звичайний правильний n -кутник;
- при $k = 2$ кожна вершина з'єднується з наступною через одну — утворюється класична зірка;
- при більших k вершини з'єднуються через більшу кількість проміжних вершин, що змінює форму та густину зіркового візерунка.

Такі правильні зіркові багатокутники позначають символом Шлефлі $\{n/k\}$. Кут повороту черепашки після кожного проведеного відрізка:

$$\alpha = \frac{360^\circ \cdot k}{n}.$$

Наприклад, для зірки $\{7/2\}$: $\alpha = \frac{360^\circ \cdot 2}{7}$.

Важливу роль відіграє найбільший спільний дільник чисел n і k :

- якщо $\gcd(n, k) = 1$ — усі n вершин належать одному замкненому контуру;
- якщо $\gcd(n, k) > 1$ — траєкторія розпадається на кілька окремих контурів, і їх кількість дорівнює $\gcd(n, k)$.

```
from math import gcd  
gcd(n, k)
```

Циклічне побудування геометричних фігур

Багато орнаментів отримують багаторазовим повторенням однієї й тієї самої фігури з поворотом після кожного повторення. Якщо фігуру потрібно побудувати n разів рівномірно навколо центра, кут повороту між сусідніми положеннями

дорівнює $\alpha = \frac{360^\circ}{n}$:

```
for i in range(n):  
    # побудова фігури  
    turtle.left(360 / n)
```

Якщо кожна наступна фігура повертається на однаковий кут, можна отримувати розети, квіти, спіралі та інші симетричні орнаменти.

Кола та правильні багатокутники

Коло заданого радіуса будують командою:

```
turtle.circle(radius)
```

За допомогою параметра `steps` замість гладкого кола можна побудувати правильний багатокутник, вписаний у це коло:

```
turtle.circle(radius, steps=6) # правильний шестикутник
```

Доповнення. Знак `radius` визначає напрямок обходу: додатне значення — проти годинникової стрілки, від'ємне — за годинниковою стрілкою. Змінюючи початкове положення та напрямок черепашки, можна керувати положенням і орієнтацією багатокутника.

Циклічна зміна кольорів

Для повторюваної зміни значень зручно використовувати остачу від ділення `%`. Якщо задано список кольорів:

```
colors = ["gold", "crimson", "lime", "cyan"]
```

то вираз `colors[i % len(colors)]` на ітерації i послідовно вибиратиме елементи з індексами 0, 1, 2, 3, 0, 1, 2, 3, ... Використання `len(colors)` замість жорстко заданого числа робить код коректним при будь-якій довжині списку.

Керування швидкістю та відображенням черепашки

Анімація кожного окремого руху може суттєво сповільнювати побудову складних зображень. Для максимальної швидкості:

```
turtle.speed(0)
```

Після завершення малювання саму черепашку (трикутний вказівник) можна приховати:

```
turtle.hideturtle()
```

Ці команди особливо корисні під час побудови орнаментів із великою кількістю елементів.

Тригонометричні функції та полярні координати

Положення точки на площині можна задавати декартовими координатами (x, y) або полярними координатами (r, θ) . Перехід від полярних координат до декартових:

$$x = r \cos \theta, \quad y = r \sin \theta.$$

Тригонометричні функції модуля `math` працюють із кутами в **радіанах**:

```
import math
math.sin(theta)
math.cos(theta)
```

Для переведення кута з градусів у радіани:

```
theta = math.radians(angle)
```

Полярне рівняння $r = f(\theta)$ перетворюють на набір точок (x, y) , послідовно обчислюючи r , x та y для різних значень θ . З'єднуючи послідовні точки лінією, отримують графік відповідної кривої.

Функції та декомпозиція графічних програм

Складне зображення доцільно розділяти на окремі складові, кожна з яких будується власною функцією, наприклад:

```
def draw_polygon(t, points, color):
    ...
```

де `t` — об'єкт черепашки, `points` — список координат вершин, а `color` — колір фігури:

```
points = [(-100, 0), (0, 100), (100, 0), (0, -100)]
```

Такий підхід називають **декомпозицією**: складну задачу поділяють на простіші частини, кожній з яких відповідає окрема функція. Під час побудови багатoshарових зображень варто дотримуватись порядку від заднього плану до переднього: спочатку тло та віддалені об'єкти, потім об'єкти середнього плану, і нарешті — елементи переднього плану.

Розділ 2. Інтерактивна графіка та анімація

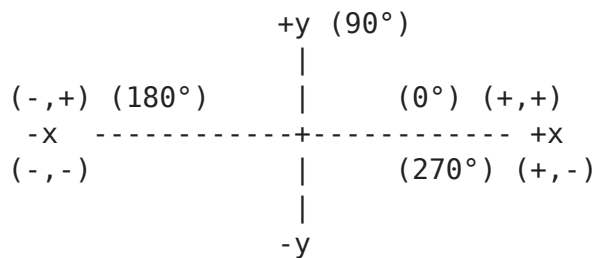
У базових скриптах команди викликаються у модульному стилі (`turtle.forward(...)`). Для розробки динамічних застосунків, які мають кілька рухомих об'єктів або реагують на події, використовується **об'єктний підхід**:

```
screen = turtle.Screen() # графічне вікно
t = turtle.Turtle()      # екземпляр об'єкта черепашки
```

Усі подальші дії виконуються як методи створеного об'єкта: `t.forward(...)`, `t.goto(...)` тощо. Об'єктний підхід є обов'язковим при керуванні кількома графічними елементами або при обробці подій (`screen.onkey`, `screen.onclick`, `screen.ontimer`).

Система координат, геометрія та малювання

Графічне вікно використовує декартову систему координат. Початок координат (0, 0) знаходиться в центрі екрана; вісь x спрямована праворуч, а вісь y — вгору:



Положення задають командою `t.goto(x, y)`, а поточні координати читають через `t.xcor()` і `t.ycor()`. Кутовий напрямок за замовчуванням (0° — вздовж осі $+x$) задається методом `t.setheading(angle)`.

Базові методи та заповнення фігур

Для зміни позиції пера без малювання слід тимчасово підняти його:

```
t.penup()
t.goto(x, y)
t.pendown()
```

Для багатокутника послідовно переходять між точками. Замкнені області зафарбовуються за допомогою пари методів `t.begin_fill()` та `t.end_fill()`:

```
t.fillcolor("orange")
t.begin_fill()
t.goto(0, 0)
t.goto(100, 0)
t.goto(50, 80)
t.goto(0, 0)
t.end_fill()
```

Для побудови дуг, півкіл чи округлих елементів (хмар, хвиль, силуетів) застосовують:

```
t.circle(radius, extent) # radius – радіус, extent – кут дуги в градусах
```

Для виведення тексту використовують `t.write("Текст", align="center", font=("Arial", 14, "bold"))`. Для прискорення малювання та приховування курсора застосовують `t.speed(0)` та `t.hideturtle()`.

Структурування сцени: Списки координат, декомпозиція та Z-index

Списки координат та кортежі

Вершини багатокутників чи складних ліній зручно зберігати у вигляді списків кортежів:

```
points = [(-300, -100), (-200, 100), (-100, -20), (0, 130), (150, -100)]

for x, y in points:
    t.goto(x, y)
```

Декомпозиція сцени

Складну сцену розбивають на окремі модульні функції. Метод декомпозиції дозволяє відокремити побудову статичних елементів від динамічних (наприклад: `draw_background()`, `draw_landscape()`, `draw_ui()`). Загальний принцип: **складна сцена = набір простих об'єктів**.

Пошарова структура (Z-index)

Порядок виклику функцій малювання визначає перекриття елементів: об'єкти, намальовані пізніше, перекривають раніше виведені шари (Задній план → Основний пейзаж → Передній план → Динамічні/Інтерактивні елементи).

Позиціонування за полярними координатами

Для рівномірного розташування об'єктів навколо центра $(0, 0)$ використовують перетворення з полярної системи координат у декартову:

1. Якщо коло розбито на N рівних частин, кут k -го елемента: $\alpha_k = k \cdot \frac{360^\circ}{N}$.
2. Кут у математичній системі (зсув від осі $+x$ проти годинникової стрілки):
 $\phi_k = 90^\circ - \alpha_k$.
3. Координати об'єкта на радіусі R :

$$x = R \cdot \cos(\phi_k)$$

$$y = R \cdot \sin(\phi_k)$$

У Python тригонометричні функції `math.cos()` і `math.sin()` приймають значення в радіанах (`math.radians(angle)`).

Об'єкти зі станом, модель руху та межі екрана

Для анімації зберігають поточний стан об'єктів (координати, розміри, швидкість, колір) у словниках або списках словників:

```
entity = {"x": 100, "y": 150, "size": 1.0, "speed": 2}
```

```
entities = [  
    {"x": 100, "y": 150, "size": 1.0, "speed": 2},  
    {"x": -50, "y": 200, "size": 0.7, "speed": 3},  
]
```

Модель руху та зациклення координат

Дискретна модель рівномірного прямолінійного руху оновлює позицію на кожному кадрі:

```
entity["x"] += entity["speed"]
```

Якщо об'єкт виходить за межі екрана, його координати переносять на протилежний бік для створення ефекту нескінченного руху:

```
if entity["x"] > right_border:  
    entity["x"] = left_border
```

Плавний зсув від залежних величин

Коли один параметр (наприклад, кут повороту елемента або позиція) залежить від кількох рівнів часових чи просторових величин, застосовують суму основного та дробового зсуву:

$$\theta = \text{Основна_величина} \cdot \Delta\theta_{\text{осн}} + \text{Дробова_частина} \cdot \Delta\theta_{\text{дроб}}$$

Приклад математичного розрахунку кутів при розбитті дискретних циклів:

- Одиниця малого циклу (1/60 кола): $\Delta\theta = \frac{360^\circ}{60} = 6^\circ$.
- Вплив дробової частини малого циклу на середній: $1 \text{ крок} = \frac{6^\circ}{60} = 0.1^\circ$.
- Формула плавної позиції: $\theta_{\text{середній}} = \text{Value}_{\text{осн}} \cdot 6^\circ + \text{Value}_{\text{дроб}} \cdot 0.1^\circ$.

Анімаційний цикл та подвійна буферизація (`tracer`, `update`, `ontimer`)

За замовчуванням кожна операція малювання відображається миттєво, що викликає миготіння та перевантажує процесор.

Для створення плавної анімації використовують буферизацію:

- `screen.tracer(0)` — вимикає автоматичне відмальовування. Усі маніпуляції виконуються в пам'яті.
- `screen.update()` — миттєво виводить сформований кадр на екран.

Головний неблокуючий цикл анімації формується рекурсивним викликом `screen.ontimer(function, delay)`:

```
def main_loop():
    # 1. Оновлення стану та координат об'єктів
    # 2. Очищення та перемальовування динамічних шарів

    screen.update()
    screen.ontimer(main_loop, 30) # Виклик наступного кадру через
30 мс (~33 FPS)

main_loop()
screen.mainloop() # Утримує вікно активним і запускає обробку
подій
```

Модулі системного часу та форматування (`time`, `datetime`)

Якщо робота анімації прив'язана до системних годинників, використовують модулі `time` та `datetime`:

- `time.localtime()` : повертає об'єкт `struct_time`, зручний для розпакування числових компонентів:

```
import time

now = time.localtime()
hours, minutes, seconds = now.tm_hour, now.tm_min, now.tm_sec
```

- `datetime.now()` : використовується для форматування рядків дати та отримання локалізованих назв:

```

from datetime import datetime

now = datetime.now()
date_str = now.strftime("%d.%m.%Y") # Наприклад: '28.09.2026'
day_name = now.strftime("%A")      # Наприклад: 'Monday'

```

Керування станом програми та обробка подій (State Management)

Інтерактивні програми реагують на дії користувача за допомогою **функцій зворотного виклику (callbacks)**. Загальний стан програми зберігається у глобальних змінних або прапорцях (State Management):

```

# Змінні стану програми
is_active_mode = False
theme_index = 0
THEMES_COUNT = 3
dynamic_objects = [] # Динамічний список об'єктів

```

Обробка кнопок миші та позиціонування

Передача координат `(x, y)` у функцію дозволяє створювати нові об'єкти в місці клацання:

```

def handle_left_click(x, y):
    global dynamic_objects
    # Створення нового об'єкта в точці клацання
    new_obj = {"x": x, "y": y, "speed": 1.5}
    dynamic_objects.append(new_obj)

def handle_right_click(x, y):
    global is_active_mode
    is_active_mode = not is_active_mode

# Реєстрація обробників миші:
screen.onclick(handle_left_click, btn=1) # Ліва кнопка миші
screen.onclick(handle_right_click, btn=3) # Права кнопка миші (у
macOS/Linux може бути btn=2)

```

Обробка клавіатурних подій та циклічне перемикання

Для циклічного перебору режимів чи тем зручно використовувати оператор залишку від ділення `%`:

```

def next_theme():
    global theme_index
    theme_index = (theme_index + 1) % THEMES_COUNT

def prev_theme():
    global theme_index
    theme_index = (theme_index - 1) % THEMES_COUNT

```

```
screen.listen() # Переведення екрана в режим "слухання" клавіатури

# Прив'язка кількох клавіш до одного обробника
screen.onkey(next_theme, "Up")
screen.onkey(next_theme, "Right")
screen.onkey(prev_theme, "Down")
screen.onkey(prev_theme, "Left")
```

Загальна архітектурна структура інтерактивної програми

1. Імпорт модулів (turtle, math, time, datetime)
2. Ініціалізація графічного вікна та налаштування tracer(0)
3. Визначення змінних стану (флаги режимів, теми, списки об'єктів)
4. Реалізація функцій малювання графічних елементів та шарів
5. Функції обробники подій миші (onclick) та клавіатури (onkey)
6. Реєстрація слухачів подій (listen, onclick, onkey)
7. Головна функція оновлення анімації (зміна даних -> малювання -> screen.update())
8. Запуск таймера (screen.ontimer) та точки входу в цикл подій (screen.mainloop())

Підсумкова таблиця інструментів

Поняття / Метод	Застосування
screen.tracer(0) / screen.update()	Подвійна буферизація кадрів для усунення мерехтіння
screen.ontimer()	Створення неблокуючого рекурсивного циклу анімації
screen.onclick(btn=1 / btn=3)	Обробка клацань миші (ліва/права кнопка) з координатами (x, y)
screen.onkey() / screen.listen()	Обробка натискання клавіш клавіатури
append() у списках	Динамічне додавання нових об'єктів під час виконання програми
% (залишок від ділення)	Циклічний кільцевий перебір тем, режимів та індексів
t.goto(), t.setheading()	Позиціонування та орієнтація графічних елементів
begin_fill() / end_fill()	Заповнення кольором замкнених контурів
math.cos() / math.sin()	Тригонометричний розрахунок координат x, y по колу
time / datetime	Отримання системного часу, компонентів tm_hour та форматування strftime

Розділ 3. Фізика абсолютно пружних зіткнень

Абсолютно пружне зіткнення двох тіл

Розглянемо одновимірне зіткнення двох тіл масами m_1 і m_2 . До зіткнення вони рухаються зі швидкостями v_1 і v_2 , а після зіткнення — зі швидкостями u_1 і u_2 . Для абсолютно пружного зіткнення виконуються два закони збереження.

Закон збереження імпульсу:

$$m_1 v_1 + m_2 v_2 = m_1 u_1 + m_2 u_2.$$

Закон збереження кінетичної енергії:

$$\frac{m_1 v_1^2}{2} + \frac{m_2 v_2^2}{2} = \frac{m_1 u_1^2}{2} + \frac{m_2 u_2^2}{2}.$$

Розв'язавши цю систему відносно швидкостей після зіткнення, отримують:

$$u_1 = \frac{m_1 - m_2}{m_1 + m_2} v_1 + \frac{2m_2}{m_1 + m_2} v_2,$$
$$u_2 = \frac{2m_1}{m_1 + m_2} v_1 + \frac{m_2 - m_1}{m_1 + m_2} v_2.$$

Ці формули визначають швидкості обох тіл після кожного зіткнення.

Пружне зіткнення з нерухомою стіною

Стіну вважають тілом нескінченно великої маси, яке саме не рухається. Формально це відповідає границі $m_2 \rightarrow \infty$ в наведених вище формулах, звідки $u_1 \rightarrow -v_1$: швидкість тіла змінює знак, а її модуль залишається незмінним. Наприклад, якщо тіло рухається до стіни зі швидкістю $v = -3$ м/с, то після зіткнення воно рухатиметься зі швидкістю $u = 3$ м/с.

Оскільки $|u| = |v|$, кінетична енергія тіла при відбитті від стіни не змінюється — це видно й безпосередньо з формули $E_k = \frac{1}{2} m v^2$, яка залежить лише від модуля швидкості.

Рух між зіткненнями та момент наступної події

Якщо на тіло не діють сили, воно рухається рівномірно:

$$x(t) = x_0 + vt,$$

де x_0 — початкова координата, v — швидкість, t — час. Тому, знаючи координати й швидкості двох кульок, можна аналітично знайти момент їхнього наступного зіткнення, не перебираючи час малими кроками.

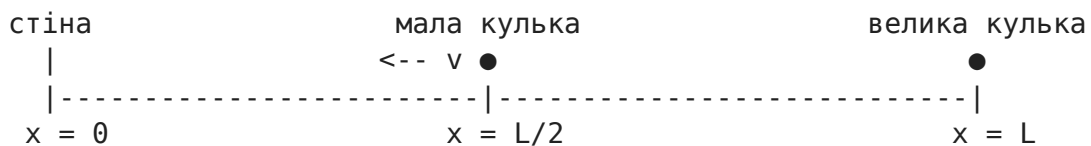
Якщо мала кулька перебуває в точці x_1 зі швидкістю v_1 , а велика — у точці x_2 зі швидкістю v_2 , момент зустрічі t знаходять із рівняння

$$x_1 + v_1 t = x_2 + v_2 t \implies t = \frac{x_2 - x_1}{v_1 - v_2},$$

за умови, що знайдене $t > 0$ і тіла дійсно рухаються назустріч одне одному (тобто відстань між ними справді скорочується). Аналогічно момент зіткнення малої кульки зі стіною ($x = 0$) знаходять із рівняння $x_1 + v_1 t = 0$, тобто $t = -x_1/v_1$ (має сенс лише коли $v_1 < 0$, тобто кулька рухається до стіни).

Подієво-орієнтоване моделювання. Такий підхід — обчислення моменту наступної події безпосередньо з рівнянь руху, а не покроковим перебором часу з кроком dt — називають *подієво-орієнтованим* (event-driven) моделюванням. Він точний і не накопичує похибку, характерну для чисельного інтегрування з фіксованим кроком.

Система «стіна — мала кулька — велика кулька»



Мала кулька має масу m і початкову швидкість v , спрямовану до стіни. Велика кулька має масу M і спочатку нерухома. Початкові умови:

$$x_m(0) = \frac{L}{2}, \quad x_M(0) = L, \quad v_m(0) = -v, \quad v_M(0) = 0.$$

Після зіткнення кульок швидкості змінюються за формулами пружного зіткнення; після зіткнення малої кульки зі стіною її швидкість змінює знак. Тому виникає послідовність подій:

зіткнення кульок $\rightarrow$ зіткнення зі стіною $\rightarrow$ зіткнення кульок $\rightarrow \dots$

На кожному кроці потрібно визначати нові швидкості тіл і час до наступної події.

Умова завершення процесу

Нехай мала кулька перебуває ліворуч від великої: $x_m < x_M$. Якщо мала кулька рухається вправо ($v_m > 0$), але її швидкість не перевищує швидкості великої кульки ($v_m \leq v_M$), вона більше не зможе наздогнати велику кульку — нових зіткнень не буде, і моделювання можна завершити.

Пройдений шлях

Пройдений шлях не слід плутати з переміщенням: якщо тіло рухалося спочатку вправо, а потім вліво, переміщення може бути малим, а шлях враховує всю довжину траєкторії. Для чисельного накопичення шляху на кожному етапі використовують

$$S_{\text{new}} = S_{\text{old}} + |\Delta x|,$$

де S — повний шлях, пройдений малою кулькою.

Практичні завдання

Завдання 1 (див. рисунки внизу завдання)

1.1 Малювання правильної зірки

Напишіть функцію `draw_star(n, length, k=2)`, яка буде малювати правильну зірку з n вершинами суцільною лінією. Довжина кожного відрізка має дорівнювати `length`, а параметр `k` визначає крок переходу від однієї вершини до іншої. Кут повороту черепашки обчислюйте за формулою $\text{angle} = \frac{360^\circ k}{n}$. Якщо $\text{gcd}(n, k) \neq 1$, виведіть у консоль попередження про те, що траєкторія може складатися з кількох окремих контурів. Для побудови використайте цикл із n ітерацій. Зробіть зірку зафарбованою, застосувавши `begin_fill()` та `end_fill()`. Для перевірки створіть графічне вікно розміром 600×600 пікселів і побудуйте семикутну зірку з параметрами `length = 150` та `k = 2`.

1.2. Побудова кольорової розети

Створіть програму, яка за допомогою модуля `turtle` буде малювати візерунок із кіл на чорному тлі. Побудуйте 36 кіл радіусом 80 пікселів. Після кожного кола повертайте черепашку ліворуч на 10° . Для кожного кола змінюйте колір лінії, циклічно вибираючи його зі списку з шести кольорів. Для цього використайте операцію остачі від ділення: `colors[i % len(colors)]`. Для прискорення побудови встановіть максимальну швидкість черепашки за допомогою `speed(0)` та приховуйте її після завершення малювання. У результаті має утворитися симетрична кольорова розета з 36 кіл.

1.3. Квадратна кольорова спіраль

Створіть програму для побудови квадратної спіралі з 60 відрізків на чорному тлі. Початкова довжина відрізка має дорівнювати 10 пікселям. Після кожного кроку збільшуйте її на 4 пікселі. Після проведення кожного відрізка повертайте черепашку праворуч на 90° . Передбачте поступове збільшення товщини лінії:

- для перших 20 відрізків — `pensize(2)` ;
- для наступних 20 — `pensize(3)` ;
- для останніх 20 — `pensize(4)` .

Колір лінії змінюйте циклічно. Послідовно використовуйте золотий, малиновий, салатовий та блакитний кольори. Для прискорення побудови встановіть `speed(0)` та приховайте черепашку після завершення малювання.

1.4. Оптична квітка зі зсунутих ромбів

Створіть програму, яка будує візерунок із 24 ромбів на чорному тлі. Кожен ромб повинен мати довжину сторони 100 пікселів та внутрішні кути 60° і 120° . Після побудови кожного ромба повертайте черепашку ліворуч на 15° . Колір лінії змінюйте залежно від номера побудованого ромба:

- перші 12 ромбів — блакитні (`cyan`);
- наступні 12 — помаранчеві (`orange`).

Встановіть `speed(0)` та приховайте черепашку після завершення побудови. У результаті має утворитися симетричний двоколірний візерунок із вираженим оптичним ефектом.

1.5. Павутинка з концентричних шестикутників

Створіть програму для побудови геометричної павутинки з шістьма радіальними променями та сімома концентричними шестикутниками на чорному тлі. Спочатку побудуйте шість сірих променів довжиною 150 пікселів. Промені повинні виходити з центра та утворювати між сусідніми променями кут 60° . Потім побудуйте сім концентричних шестикутників із радіусами:

20, 40, 60, 80, 100, 120, 140. Для побудови шестикутників використовуйте `circle(size, steps=6)` , розпочинаючи побудову з точки `(0, -size)` . Для перших п'яти внутрішніх шестикутників використовуйте білі лінії товщиною 1 піксель. Для двох зовнішніх шестикутників використовуйте жовті лінії товщиною 3 пікселі. Для прискорення побудови встановіть `speed(0)` та приховайте черепашку після завершення роботи.

1.6. Візерунок «Кристал» із кіл

Створіть програму для побудови симетричного геометричного візерунка з 20 кіл на чорному тлі. Для i -го кола обчислюйте радіус за формулою $r = 30 + 5i$. Після побудови кожного кола повертайте черепашку ліворуч на 18° . Колір кола визначаєте залежно від його радіуса:

- якщо $r < 70$ — `deep sky blue` ;
- якщо $70 \leq r < 110$ — `royal blue` ;

- якщо $r \geq 110$ — `gold`.

Товщина лінії має дорівнювати 2 пікселям. Для прискорення побудови встановіть `speed(0)` та приховайте черепашку.

1.7. Кольорова шестикутна спіраль

Створіть програму для побудови спіралі з 360 відрізків на чорному тлі. На кожній ітерації переміщуйте черепашку вперед на відстань `x` пікселів і повертайте її ліворуч на 59° . Послідовно змінюйте колір лінії, циклічно вибираючи його зі списку з шести кольорів. Для цього використайте номер поточної ітерації та операцію остачі від ділення. Товщину лінії змінюйте залежно від номера ітерації за формулою $\text{pensize} = \frac{x}{100} + 1$. Для прискорення побудови встановіть `speed(0)`.

1.8. Малювання математичної троянди в полярних координатах

Створіть програму для побудови полярної кривої $r = 180 \cos(15\theta)$ на чорному тлі. Для послідовних значень кута θ від 0° до 360° обчислюйте радіус r , а потім декартові координати точки за формулами: $x = r \cos \theta$, $y = r \sin \theta$.

Тригонометричні функції модуля `math` використовують радіани, тому перед обчисленням синуса та косинуса переведіть значення θ з градусів у радіани. Колір лінії змінюйте залежно від абсолютного значення радіуса:

- якщо $|r| > 100$, використовуйте `cyan`;
- в інших випадках — `magenta`.

Послідовні точки сполучайте суцільною лінією товщиною 2 пікселі за допомогою `goto(x, y)`. Для прискорення побудови встановіть `speed(0)` та приховайте черепашку.

1.9. Трисекторне сонце з кільцями

Створіть програму для побудови променистої фігури із 72 променів на чорному тлі. Для кожного променя:

1. перемістіть черепашку вперед на 120 пікселів;
2. на кінці променя намалюйте коло радіусом 15 пікселів;
3. поверніться до центра за допомогою `backward(120)`;
4. поверніть черепашку ліворуч на 5° .

Розділіть промені на три групи по 24:

- перші 24 — золотого кольору (`gold`);
- наступні 24 — малинового (`crimson`);

- останні 24 — бірюзового (`turquoise`).

Для прискорення побудови встановіть `speed(0)` та приховайте черепашку.

1.10. Оптична розетка зі зсунутих квадратів

Створіть програму для побудови геометричної розетки з 72 квадратів на чорному тлі. У кожній ітерації побудуйте квадрат зі стороною 150 пікселів, після чого поверніть черепашку ліворуч на 5° . Колір контуру змінюйте циклічно залежно від номера ітерації. Послідовно використовуйте:

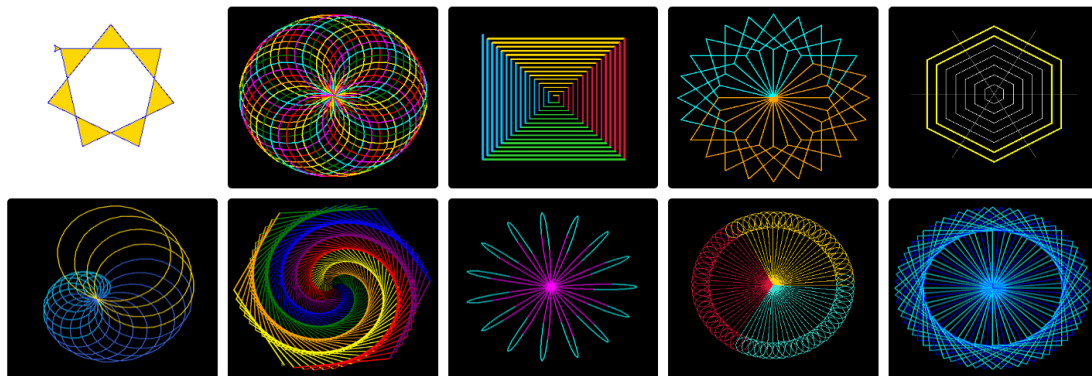
- `cyan` ;
- `deep sky blue` ;
- `blue` .

Товщина лінії має дорівнювати 2 пікселям. Для прискорення побудови встановіть `speed(0)` та приховайте черепашку після завершення роботи. У результаті має утворитися симетрична багат шарова розетка з плавною зміною кольорів.

Ось цілісний варіант завдання, де поєднано **векторну графіку, функції, структури даних, цикли, анімацію та інтерактивну взаємодію з користувачем**. Я б саме таке формулювання дав студентам як підсумкове творче завдання.

1.11. Відтворення пейзажу за допомогою декомпозиції

Створіть програму Python, яка відтворює векторний гірський пейзаж шляхом декомпозиції зображення на окремі зафарбовані шари (небо, сонце, гірські хребти, водойма, передній план та птахи). Усі складні фігури малюйте послідовно від заднього плану до переднього через універсальну функцію `draw_polygon(t, points, color)`, яка приймає список координат `[(x, y), ...]` і використовує `begin_fill()` та `end_fill()`. Для птахів створіть окрему функцію `draw_bird(t, x, y, size)`, що будує силует за допомогою двох закруглених дуг `circle(radius, extent)`. Обов'язково налаштуйте швидкість `speed(0)` та приховайте черепашку `hideturtle()`. Підберіть кольори для кожного елемента (персикове небо, жовте сонце, помаранчеві й теракотові гори, пурпурова вода, темно-фіолетові скелі переднього плану). У результаті повинна утворитися гармонійна багат шарова композиція з чіткою геометрією.



Завдання 2 (див.рисунок внизу завдання)

Створіть інтерактивну програму, яка реалізує функціональний годинник з розширеним управлінням та візуальними ефектами.

Технічні вимоги:

1. Аналоговий режим (початковий):

- Малювання круглого циферблата з 12 основними поділками та числовими позначками (1-12).
- Трьохстрілковий механізм (годинна, хвилинна, секундна). Стрілки мають відрізнятися товщиною, довжиною та кольором.

- **Плавний рух стрілок:** годинна та хвилинна стрілки повинні розраховуватися за розширеними формулами з урахуванням хвилин та секунд.

2. Інтерактивні режими та обробка миші:

- **Клік ЛІВОЮ кнопкою миші:** перемикає вигляд між **аналоговим** та **цифровим** (великий світлодіодний/неоновий текст у форматі `HH:MM:SS`).
- **Клік ПРАВОЮ кнопкою миші:** вмикає/вимикає додаткове поле з **поточною датою та днем тижня** (наприклад: `28.09.2026` | `Понеділок`) у нижній частині екрана.

3. Колірні теми (управління клавішами-стрілками):

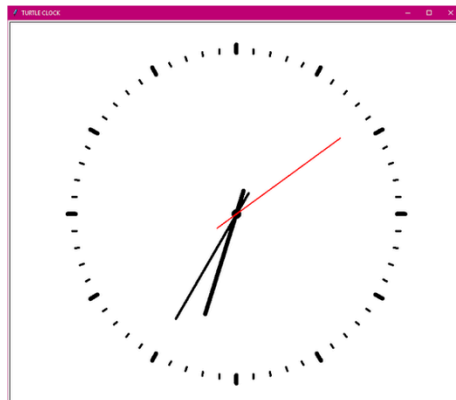
- Реалізуйте масив мінімум з **3-х тем** (наприклад: *Денна*, *Нічна*, *Cyberpunk/Neon*).
- Клавіші **Up** та **Right** перемикають тему на наступну.
- Клавіші **Down** та **Left** перемикають тему на попередню.
- Тема повинна змінювати: колір фону, циферблата, кожної зі стрілок та текстових блоків.

4. Візуальний сигнал (початок хвилини):

- Коли секунди дорівнюють `00` , програма повинна подавати короткий візуальний сигнал (наприклад, фоновий спалах екрана або зміна кольору рамки).

5. Оптимізація кадру:

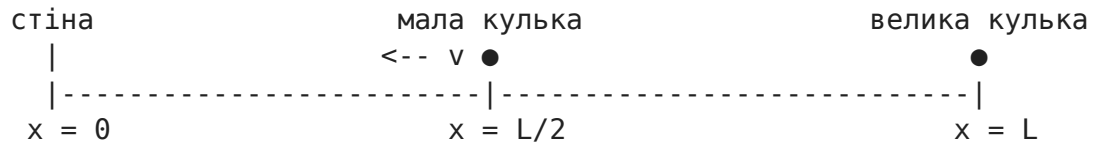
- Повна відсутність миготіння завдяки зв'язці `screen.tracer(0)` , `screen.update()` та `screen.ontimer()` .



Завдання 3. Моделювання послідовності пружних зіткнень

Розгляньте систему, що складається з нерухомої стіни та двох кульок. На відстані L від стіни знаходиться нерухома кулька маси M . Між стіною та цією кулькою, у точці $L/2$, розташована інша кулька маси m . У початковий момент мала кулька починає рухатися зі швидкістю v у напрямку стіни.

Схематично:



Взаємодія малої кульки зі стіною та взаємодія між двома кульками вважаються **абсолютно пружними**. Необхідно дослідити рух системи та реалізувати його комп'ютерне моделювання.

Вхідні параметри

Програма повинна отримувати від користувача такі параметри:

- m — маса малої кульки;
- M — маса великої кульки;
- L — початкова відстань від стіни до великої кульки;
- v — початкова швидкість малої кульки.

Початкові координати та швидкості визначаються умовами:

$$x_m(0) = \frac{L}{2}, \quad x_M(0) = L,$$

$$v_m(0) = -v, \quad v_M(0) = 0.$$

Моделювання руху

Реалізуйте програму, яка моделює весь процес руху кульок. Програма повинна:

1. визначати наступну подію — зіткнення малої кульки зі стіною або зіткнення двох кульок;
2. визначати момент цієї події;
3. переміщувати кульки до моменту зіткнення;
4. змінювати швидкості після зіткнення;
5. продовжувати моделювання;
6. визначати момент, коли нових зіткнень більше не буде;
7. обчислювати повний шлях, пройдений малою кулькою.

Не використовуйте просте моделювання з фіксованим малим кроком часу dt . Необхідно визначати моменти зіткнень безпосередньо з рівнянь руху.

Анімація за допомогою `turtle`

За допомогою бібліотеки `turtle` створіть анімацію руху системи. На екрані повинні бути представлені:

- стіна;
- мала кулька;
- велика кулька;
- напрямок їхнього руху;
- бажано — поточні значення швидкостей.

В анімації повинні бути помітні всі зіткнення:

- малої кульки зі стіною;
- малої та великої кульок між собою.

Швидкість відтворення анімації можна масштабувати для зручності спостереження.

Дослідження залежності шляху від параметрів

Позначимо повний шлях, пройдений малою кулькою до завершення процесу зіткнень (процес зіткнень вважається завершеним в момент останнього удару об стінку), через S . Дослідіть залежність S від кожного з чотирьох початкових параметрів.

Залежність від маси малої кульки

Зафіксуйте M , L та v . Побудуйте графік $S = S(m)$. Використайте щонайменше 20 різних значень m .

Залежність від маси великої кульки

Зафіксуйте m , L та v . Побудуйте графік $S = S(M)$. Використайте щонайменше 20 різних значень M .

Залежність від відстані

Зафіксуйте m , M та v . Побудуйте графік $S = S(L)$. Використайте щонайменше 20 різних значень L .

Залежність від початкової швидкості

Зафіксуйте m , M та L . Побудуйте графік $S = S(v)$. Використайте щонайменше 20 різних значень v . Особливо важливо пояснити, **чому отримана залежність має саме такий вигляд**.

Дослідження кількості зіткнень

Для кожного проведеного моделювання визначте також кількість зіткнень: N .

Дослідіть залежність кількості зіткнень від співвідношення мас $r = \frac{M}{m}$. Побудуйте графік $N = N(r)$. Для дослідження використовуйте широкий діапазон значень r .

Фізичний аналіз

Для кожного отриманого графіка потрібно не лише описати його вигляд, а й пояснити результат із погляду фізики.

Зокрема, дайте відповіді на питання:

1. Як маса малої кульки впливає на її повний шлях?
2. Як маса великої кульки впливає на кількість зіткнень?
3. Як зміна L впливає на повний шлях?
4. Як впливає початкова швидкість v ?
5. Що відбувається, коли $M = m$?
6. Що відбувається при $M \gg m$?
7. Чому зіткнення зі стіною не змінює кінетичної енергії малої кульки?
8. Чому при зіткненні двох кульок змінюються швидкості обох тіл?

Вимоги до програми

Програма повинна:

- використовувати функції;
- використовувати `turtle` для анімації;
- використовувати `matplotlib` для графіків;
- самостійно реалізовувати фізичну модель;
- використовувати закони збереження імпульсу та енергії;
- перевіряти коректність введених параметрів;
- мати зрозумілу структуру;
- містити коментарі до основних частин алгоритму;
- будувати графіки щонайменше за 20 значеннями параметра;
- містити підписи осей та заголовки графіків.